

Contents

Smart Samarpan Academy - Interview Preparation (Part 6 to 9)	1
PART 6 — FEATURE EXECUTION FLOW	1
PART 7 — CODE UNDERSTANDING	4
PART 8 — DATABASE (MongoDB)	6
PART 9 — API LIST	7

Smart Samarpan Academy - Interview Preparation (Part 6 to 9)

Welcome to Part 6 to 9 of the interview preparation guide for the Smart Samarpan Academy project. Ye guide aapko project ke har ek feature ka execution flow, code ki deep understanding, database architecture aur APIs ke baare mein detail mein samjhayega.

PART 6 — FEATURE EXECUTION FLOW

Yahan hum har major feature ka complete flow trace karenge ki ek user ke click karne se lekar database mein save hone aur wapas frontend par result aane tak kya kya hota hai.

1. User Registration + OTP Verification

- **USER ACTION:** Fills out registration form and clicks Register.
- **FRONTEND:** Register.jsx calls registerUser from useContext.jsx.
- **API REQUEST:** POST /api/user/register with {name, email, password}.
- **BACKEND ROUTE:** server/routes/user.js routes to register controller.
- **CONTROLLER/LOGIC:** server/controllers/user.js. Checks if user exists. Hashes password with bcrypt. Generates 6-digit OTP. Creates activation JWT (valid for 5m). Sends OTP email via sendMail().
- **RESPONSE:** Returns activationToken and success message.
- **FRONTEND:** Redirects user to Verify.jsx.
- **USER ACTION:** Enters OTP and clicks Verify.
- **FRONTEND:** Verify.jsx calls verifyOtp from useContext.jsx.
- **API REQUEST:** POST /api/user/verify with {otp, activationToken}.
- **CONTROLLER/LOGIC:** verifyUser controller decodes token, compares OTP. If matches, creates User in DB.
- **DATABASE:** Inserts new document in users collection.
- **RESPONSE:** Success message. Frontend redirects to Login.

2. User Login (Email/Password)

- **USER ACTION:** Enters email & password, clicks Login.
- **FRONTEND:** Login.jsx calls loginUser in useContext.jsx.
- **API REQUEST:** POST /api/user/login with {email, password}.
- **CONTROLLER/LOGIC:** Finds user by email. Uses bcrypt.compare to check password. Generates JWT token (JWT_SECRET, 15d expiry).
- **DATABASE:** Reads from users collection.
- **RESPONSE:** Returns {message, token, user}.
- **FRONTEND:** Saves token to local storage, fetches user, redirects to Home/Dashboard.

3. Google OAuth Login

- **USER ACTION:** Clicks "Sign in with Google".
- **FRONTEND:** Google popup gives ID token. Sends it to backend.
- **API REQUEST:** POST /api/user/google-login with {token}.
- **CONTROLLER/LOGIC:** Verifies Google ID token using `client.verifyIdToken`. Gets user email, name, picture. If user doesn't exist, creates one with a dummy hashed password. Generates JWT.
- **DATABASE:** Reads/Writes users collection.
- **RESPONSE:** Returns {token, user}. Frontend logs user in.

4. Forgot/Reset Password

- **USER ACTION:** Clicks Forgot Password, enters email.
- **API REQUEST:** POST /api/user/forgot -> forgotPassword controller sends reset link via email.
- **USER ACTION:** Clicks link in email, goes to ResetPassword.jsx.
- **API REQUEST:** POST /api/user/reset?token=... -> resetPassword updates password in DB.

5. Browse Courses

- **USER ACTION:** Visits /courses page.
- **FRONTEND:** Courses.jsx calls fetchCourses from CourseContext.
- **API REQUEST:** GET /api/course/all.
- **CONTROLLER/LOGIC:** getAllCourses fetches all courses from DB.
- **DATABASE:** Courses collection read.
- **RESPONSE:** Array of courses -> mapped to CourseCard components.

6. Buy a Course (Razorpay Payment)

- **USER ACTION:** Clicks "Buy Now" on CourseDescription.jsx.
- **API REQUEST 1:** POST /api/course/checkout/:id.
- **CONTROLLER 1:** checkout controller. Creates Razorpay order instance.orders.create({amount, currency: 'INR'}). Returns order details.
- **FRONTEND:** Opens Razorpay checkout popup. User pays.
- **API REQUEST 2:** On success, calls POST /api/verification/:id with Razorpay signature details.
- **CONTROLLER 2:** paymentVerification verifies HMAC SHA256 signature.
- **DATABASE:** Saves Payment, pushes courseId to User.subscription, creates Progress document.
- **RESPONSE:** Success, redirects to Dashboard.

7. Watch a Lecture + Mark Complete

- **USER ACTION:** Opens a lecture video in Lecture.jsx.
- **FRONTEND:** Fetches lecture details. Video plays. On video end, marks complete.
- **API REQUEST:** POST /api/user/progress?course=...&lectureId=....
- **CONTROLLER/LOGIC:** addProgress checks if lecture is already in completedLectures. If not, pushes it.
- **DATABASE:** Updates Progress collection.

8. AI Quiz Generation + Submit Results

- **USER ACTION:** Selects topic, difficulty in QuizGenerator.jsx and clicks Generate.
- **API REQUEST:** POST /api/generate-quiz with {topic, difficulty, num_questions}.

- **CONTROLLER/LOGIC:** Builds prompt, calls Gemini API via `fetchWithRetry`. Parses Mark-down/JSON response.
- **RESPONSE:** Array of quiz objects (questions, options, `correct_answer`).
- **USER ACTION:** Answers quiz and submits.
- **API REQUEST:** `POST /api/submit-quiz-result`.
- **DATABASE:** Saves `QuizResult` in DB.

9. AI Formula Generation

- **USER ACTION:** Requests formulas for a topic.
- **API REQUEST:** `POST /api/generate-formulas`.
- **CONTROLLER/LOGIC:** Calls Gemini to generate KaTeX formulas.
- **RESPONSE:** Returns formula data. Frontend renders using KaTeX.

10. AI Performance Analysis

- **USER ACTION:** Clicks “Performance Analysis” in Dashboard.
- **API REQUEST:** `GET /api/user/performance-analysis`.
- **CONTROLLER/LOGIC:** Fetches all `QuizResult` for user. Sends data to Gemini for analysis.
- **RESPONSE:** Gemini returns text report and chart data (pie chart stats).

11. Admin: Create Course

- **ADMIN ACTION:** Admin goes to Add Course, uploads image, fills details.
- **FRONTEND:** Submits `FormData`.
- **API REQUEST:** `POST /api/course/new` (with `multer`).
- **CONTROLLER/LOGIC:** `isAuth`, `isAdmin` middleware pass. `createCourse` saves course with uploaded image path.

12. Admin: Add Lecture

- **ADMIN ACTION:** Uploads video in specific course.
- **API REQUEST:** `POST /api/course/:id` (with `multer`).
- **CONTROLLER/LOGIC:** `addLectures` saves Lecture and links it to course.

13. Admin: Delete Course

- **ADMIN ACTION:** Clicks Delete Course.
- **API REQUEST:** `DELETE /api/course/:id`.
- **CONTROLLER/LOGIC:** Deletes course, related lectures, and removes from users’ subscriptions.

14. Admin: Manage Users

- **SUPERADMIN ACTION:** Changes a user’s role to admin.
- **API REQUEST:** `PUT /api/user/:id`.
- **CONTROLLER/LOGIC:** `updateRole` toggles role between ‘user’ and ‘admin’.

15. Progress Tracking

- **USER ACTION:** Views course in Dashboard.
- **API REQUEST:** `GET /api/user/progress?course=...`
- **CONTROLLER/LOGIC:** `getYourProgress` counts completed lectures vs total lectures. Returns percentage.

PART 7 — CODE UNDERSTANDING

Interviewer aksar project ke core functions likhne ya explain karne ko bolte hain. Ye sections bahut important hain.

HIGH PRIORITY CODE

1. isAuth Middleware (server/middlewares/isAuth.js) **Why it exists:** To protect routes so only logged-in users can access them. **Input:** req.headers.token **Output:** Attaches user object to req.user or returns 401/500 error. **Interview Question:** “How do you secure your backend APIs?” or “Explain how JWT works in your project.”

```
import jwt from 'jsonwebtoken';
import { User } from '../models/User.js';

export const isAuth = async (req, res, next) => {
  try {
    const token = req.headers.token; // Step 1: Token nikalna
    if (!token) return res.status(403).json({ message: "Please Login" });

    const decoded = jwt.verify(token, process.env.JWT_SECRET); // Step 2: Token verify karna
    req.user = await User.findById(decoded._id); // Step 3: User find karke req object me dalna
    next(); // Step 4: Agle function (controller) pe bhej dena
  } catch (error) {
    res.status(500).json({ message: "Login First" });
  }
}
```

2. register function (server/controllers/user.js lines 14-35) **Why it exists:** To temporarily store user data and send OTP before actual account creation. **Input:** {name, email, password} **Output:** Sends email, returns activationToken. **Interview Question:** “Batao user registration ka flow with email verification kaise kaam karta hai?”

```
export const register = async (req, res) => {
  try {
    const { name, email, password } = req.body;
    let user = await User.findOne({ email });
    if (user) return res.status(400).json({ message: "User Already exists" });

    const hashPassword = await bcrypt.hash(password, 10); // Password hash karna security k
    // 6-digit random OTP generate karna
    const otp = Math.floor(Math.random() * 1000000);

    // Ek temporary JWT token banana jisme user data aur OTP dono hai (valid for 5 mins)
    const activationToken = jwt.sign({
      user: { name, email, password: hashPassword },
      otp
    }, process.env.Activation_Secret, { expiresIn: "5m" });

    // Email send karna
    await sendMail(email, "Samarpan Math Academy", `Please verify your account. Your OTP is`);

    res.status(200).json({ message: "Otp sent to your mail", activationToken });
  } catch (error) {
    //
  }
}
```

```

    res.status(500).json({ message: error.message });
  }
}

```

3. paymentVerification function (server/controllers/course.js lines 91-133) **Why it exists:** Razorpay payment successful hone ke baad data verify and save karna. **Input:** Razorpay signature, order id, payment id, user id, course id. **Output:** Database update for Payment, User, and Progress. **Interview Question:** "Razorpay webhook/verification kaise handle kiya? Signature verify kyu karte hain?" (Answer: Taaki koi fake payment successful request na bhej sake).

```

export const paymentVerification = async (req, res) => {
  try {
    const { razorpay_order_id, razorpay_payment_id, razorpay_signature } = req.body;
    const body = razorpay_order_id + "|" + razorpay_payment_id;

    // Razorpay secret use karke apna signature banana
    const expectedSignature = crypto.createHmac("sha256", process.env.Razorpay_Secret)
      .update(body.toString()).digest("hex");

    const isAuthentic = expectedSignature === razorpay_signature; // Matching signature

    if (isAuthentic) {
      await Payment.create({ razorpay_order_id, razorpay_payment_id, razorpay_signature });

      const user = await User.findById(req.user._id);
      const course = await Courses.findById(req.params.id);
      user.subscription.push(course._id); // User ko course assign karna
      await user.save();

      // Progress document create karna track karne ke liye ki kitna padha
      await Progress.create({ course: course._id, completedLectures: [], user: req.user._id });

      res.status(200).json({ message: "Course Purchased Successfully" });
    } else {
      return res.status(400).json({ message: "Payment Failed" });
    }
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}

```

4. fetchWithRetry function (server/index.js lines 47-55) **Why it exists:** Gemini API kabhi kabhi rate limit (429 Error) de deta hai. Ye function thodi der ruk kar wapas try karta hai (Exponential Backoff). **Interview Question:** "How do you handle API failures or rate limits from third-party AI APIs?"

```

const fetchWithRetry = async (prompt, retries = 3, delay = 1000) => {
  for (let i = 0; i < retries; i++) {
    try {
      // Attempt to call Gemini API
      const result = await model.generateContent(prompt);
      return result.response.text();
    } catch (error) {
      if (error.status === 429 && i < retries - 1) { // 429 matlab "Too Many Requests"
        await new Promise(res => setTimeout(res, delay)); // delay time tak ruko
      }
    }
  }
}

```

```

        delay *= 2; // delay time ko double kardo (Exponential backoff)
    } else throw error;
  }
}
}

```

MEDIUM PRIORITY CODE

1. **TryCatch middleware:** server/middlewares/TryCatch.js - Ek wrapper function jo try-catch block likhne se bachata hai har controller me.
2. **addProgress function:** Array includes() method check karta hai ki kya lecture ID pehle se array me hai, agar nahi toh push kardo.
3. **deleteCourse function:** Course delete karte time dhyan rakhna hota hai ki uska photo unlink (delete) ho, aur uske saare lectures bhi delete hon database se.
4. **getPerformanceAnalysis:** Gemini API ko previous scores bhej kar insights lena. Prompt engineering yahan important hai.

PART 8 — DATABASE (MongoDB)

Project uses MongoDB (NoSQL). Why MongoDB? Kyunki MERN stack me JSON like documents (BSON) directly frontend se backend aur database me flow karte hain. Schema flexible hai.

Collections & Fields

1. **User** (models/User.js)
 - name, email, password (String)
 - role, mainrole (String, default 'user')
 - subscription: Array of ObjectId (ref Courses) - Kaun kaun se courses kharide hain.
 - resetPasswordExpire (Date)
2. **Courses** (models/Courses.js)
 - title, description, image, category, createdBy (String)
 - price, duration (Number)
3. **Lecture** (models/Lecture.js)
 - title, description, video (String)
 - course: ObjectId (ref Courses) - Kis course ka lecture hai.
4. **Payment** (models/Payment.js)
 - razorpay_order_id, razorpay_payment_id, razorpay_signature (String)
 - Audit trail of all transactions.
5. **Progress** (models/Progress.js)
 - course: ObjectId (ref Courses)
 - user: ObjectId (ref User)
 - completedLectures: Array of ObjectId (ref Lecture) - Track karta hai video complete hue ya nahi.
6. **QuizResult** (models/QuizResult.js)
 - user: ObjectId (ref User)
 - topic, difficulty (String/Enum)
 - score, totalQuestions (Number)
 - User ki performance analysis generate karne ke liye ye use hota hai.

Relationships: - **1-to-Many:** Course to Lectures. Ek course me bahut lectures hote hain. - **Many-to-Many:** Users to Courses (Users have subscriptions array). - **1-to-Many:** User to QuizResults and User to Progress.

PART 9 — API LIST

Ek nazar me aapke backend ke saare APIs:

USER APIs

- **POST** /api/user/register
 - Purpose: Account banana aur OTP bhejna
 - Request Body: {name, email, password}
- **POST** /api/user/verify
 - Purpose: OTP verify karke user save karna
 - Request Body: {otp, activationToken}
- **POST** /api/user/login
 - Purpose: Login and get JWT token
 - Request Body: {email, password}
- **POST** /api/user/google-login
 - Purpose: Google OAuth Login
 - Request Body: {token} (Google ID Token)
- **GET** /api/user/me (isAuth)
 - Purpose: Logged in user ki profile fetch karna
- **POST** /api/user/forgot
 - Purpose: Send password reset link to email
- **POST** /api/user/reset (token in query)
 - Purpose: Set new password
- **POST** /api/user/progress (isAuth)
 - Purpose: Lecture complete mark karna (query: ?course=id&lectureId=id)
- **GET** /api/user/progress (isAuth)
 - Purpose: Course ki progress % dekhna
- **GET** /api/user/performance-analysis (isAuth)
 - Purpose: Gemini AI se performance stats nikalna

COURSE APIs

- **GET** /api/course/all
 - Purpose: Saare courses list karna
- **GET** /api/course/:id
 - Purpose: Single course details
- **GET** /api/lectures/:id (isAuth)
 - Purpose: Course ke saare lectures list karna
- **GET** /api/lecture/:id (isAuth)
 - Purpose: Single video play karna
- **GET** /api/mycourse (isAuth)
 - Purpose: User ke kharide hue courses dikhana
- **POST** /api/course/checkout/:id (isAuth)
 - Purpose: Razorpay order create karna
- **POST** /api/verification/:id (isAuth)
 - Purpose: Razorpay signature verify karke access dena

ADMIN APIs

- **POST** /api/course/new (isAuth, isAdmin, uploadFiles)
 - Purpose: Naya course add karna with image

- **POST** /api/course/:id (isAuth, isAdmin, uploadFiles)
 - Purpose: Course me naya lecture video add karna
- **DELETE** /api/course/:id (isAuth, isAdmin)
 - Purpose: Course udana
- **DELETE** /api/lecture/:id (isAuth, isAdmin)
 - Purpose: Lecture udana
- **GET** /api/stats (isAuth, isAdmin)
 - Purpose: Dashboard ke liye total users, courses stats
- **GET** /api/users (isAuth, isAdmin)
 - Purpose: Saare users list karna
- **PUT** /api/user/:id (isAuth, isAdmin)
 - Purpose: User ko admin role dena (Superadmin only feature)

AI INTEGRATION APIs (in index.js)

- **POST** /api/generate-quiz
 - Purpose: Gemini se naye math questions banana. Body: {topic, difficulty, num_questions}
- **POST** /api/submit-quiz-result
 - Purpose: Quiz score database me save karna.
- **POST** /api/generate-formulas
 - Purpose: Kisi topic ke formulas Gemini se mangwana (KaTeX format me)
- **GET** /api/get-recommendations/:userId
 - Purpose: Weak topics ke basis par next topics suggest karna.